



Revision Notes: Class on Bit Manipulation and Array Problems

Introduction

This class focused on bit manipulation techniques and their applications in solving specific array problems. The class covered the theoretical understanding of bits, their operations, and practical implementations using C++ and other programming structures. Below is a comprehensive set of revision notes covering the key topics and concepts discussed during the class.

Concepts Discussed

1. Counting Set Bits in an Integer

- **Explanation:**

- A set bit in binary representation is represented by 1.
- For any integer, counting the number of set bits involves checking each bit position for the presence of 1.

- **Approach:**

1. Iterate over each of the 32 bits (for a 32-bit integer).
2. Check if the i -th bit is set using the bitwise operation: $(n \& (1 \ll i)) \neq 0$.
3. Increment a count for each set bit encountered.

- **Code Snippet:**

```
int countSetBits(int n) {  
    int count = 0;  
    for (int i = 0; i < 32; ++i) {  
        if (n & (1 << i)) {  
            ++count;  
        }  
    }  
}
```



- **Alternate Approach:**

- Using right shift (`>>`) to continuously shift the number and check the least significant bit.

2. Maximizing Bitwise AND of Pairs (Subarrays/Arrays)

- **Problem Description:**

- Given an array of numbers, the objective is to form pairs that can maximize the bitwise AND value.

- **Approach:**

1. Start by considering the most significant bit.
2. For a given bit position, count how many numbers have that bit set.
3. If the count of set bits is two or more, consider these numbers for your pair to ensure this bit is set in the final AND result.
4. Iterate from the most significant bit to the least significant bit applying this logic **【6:10+source】** .

- **Key Observations:**

- If a bit in no two numbers aligns as 1, discard that bit in further consideration.
- You can incrementally build the result by setting bits from higher to lower if conditions are met.

3. Finding Subarrays with Specific Bitwise OR Properties

- **Explanation:**

- The problem involves finding subarrays where the bitwise OR of elements reaches a certain condition, often focused on a binary scale (e.g., OR equals 1).

- **Key Points:**

- For an array with binary properties, the bitwise OR can only result in 0 or 1.
- Subarrays resulting in bitwise OR of 1 must contain at least one element with a set bit (1).



(all elements must be 0).

4. Handling Edge Cases with Zeros

- **Concept:**

- When handling arrays, particularly binary, the challenge often is dealing with blocks of 0s which impact subarray calculations.

- **Strategy:**

1. Track consecutive zeros to quickly compute how many subarrays they form.
2. Use formula $(count * (count + 1)) / 2$ to determine how many subarrays can be formed from continuous zeros **【6:10+source】**.

Conclusion

These notes covered the foundational concepts of bitwise operations and their application in solving problems centered around integers and arrays. Understanding these techniques is crucial for optimizing and accurately implementing solutions that involve bit manipulation. Regular practice of computational logic and implementation of these strategies in various contexts enhances problem-solving skills in software engineering domains.